MICHD Classification with NumPy-Only Boosted Trees

Project 1 experimental report

28 October 2026

Abstract

We implement histogram-based second-order gradient boosting using only NumPy and standard Python. On the official BRFSS data, the tuning-selected configuration obtains holdout F1 0.4446, precision 0.3715 and recall 0.5534. Ridge regression obtains F1 0.4097. All trees are trained from the provided data; no external ML package or pretrained model is used. We supply training code, six required numerical algorithms, fitted trees, split identifiers and reproducible test predictions.

1 Data and protocol

The official data contain 328,135 labeled records, 109,379 test records and 321 input features after excluding `Id`. There are 28,975 positive MICHD labels (8.83%); 44.79% of feature entries are missing. Labels are aligned by identifier and mapped from $\{-1, +1\}$ to $\{0, 1\}$. We preserve test IDs and their order in the submission.

With seed 2026, a stratified 60/20/20 split yields 196,881 training, 65,627 tuning and 65,627 holdout records. Their positive counts are 17,385, 5,795 and 5,795. Bin edges and ridge preprocessing are fitted on training rows only. Rounds, thresholds and the choice among three boosting configurations use tuning data only. The selected configuration is saved before scoring boosting models on holdout.

The same split was previously used for an external-library XGBoost experiment. Consequently, the holdout is an internal development benchmark, not a pristine external test. This NumPy experiment uses three predefined configurations and makes no post-holdout model changes. All results below are newly measured NumPy results.

2 Original NumPy implementation

At each round, let p_i be the current logistic probability and a_i the class weight. We compute

$$g_i = a_i(p_i - y_i), \qquad h_i = a_i p_i(1 - p_i).$$

For a leaf with summed gradient G and Hessian H, its regularized Newton value is $-G/(H+\lambda)$. A split is chosen by its positive gain

$$\frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right].$$

Both children must satisfy the minimum Hessian sum. Tree outputs are multiplied by the learning rate and added to the current logit.

Histogram search. We derive at most 31 quantile cuts per feature for 32 observed-value bins. Features with at most 32 distinct values use midpoints between adjacent values. Bin zero represents missing entries. NumPy `bincount` accumulates gradient and Hessian histograms, and cumulative sums evaluate thresholds. Both missing-left and missing-right assignments are tested, including missing-versus-observed splits. No third-party tree implementation was copied.

Configuration. All models use learning rate 0.1, $\lambda = 10$, minimum child Hessian sum 20, row sampling 0.7 and feature sampling 0.35 per tree. The three candidates are depth two, depth three, and depth three with positive weight three. Maximum training length is 300 rounds. Every ten rounds we maximize tuning F1 over distinct score cutoffs, grouping ties. Training stops after 60 rounds without an improvement exceeding 10^{-6}; only the best tree prefix is retained. The best boosting configuration is then selected by tuning F1.

Reference baselines. Always-negative prediction demonstrates class imbalance. Ridge regression uses training-only mean imputation, standardization and an intercept, with $\lambda \in \{10^{-4}, 10^{-2}, 1\}$ and a tuning-selected cutoff. Its selected λ is 10^{-4}. Ridge is a reference baseline, not a candidate in the boosting selection.

3 Measured results

Model	Tune F1	Holdout F1	Precision	Recall	Accuracy
Always negative	0.0000	0.0000	0.0000	0.0000	0.9117
Ridge	0.4143	0.4097	0.3541	0.4859	0.8763
NumPy trees, depth 2	0.4463	0.4420	0.3686	0.5519	0.8770
NumPy trees, depth 3	0.4483	0.4446	0.3715	0.5534	0.8779
NumPy trees, depth 3, weight 3	0.4475	0.4434	0.3663	0.5617	0.8755

Table 1: Fixed internal holdout. Each cutoff is selected on tuning data; model choice is restricted to the three boosting configurations.

The selected configuration is NumPy trees, depth 3, with 210 trees and threshold 0.209967. It improves holdout F1 over ridge by 0.0349. The selected model's default 0.5 cutoff yields F1 0.2513; threshold tuning changes this to 0.4446. This comparison uses the same fitted trees, isolating the effect of the decision rule.

The selected model has 3,207 true positives, 5,425 false positives, 2,588 false negatives and 54,407 true negatives. Its accuracy is 0.8779. The majority-negative baseline achieves 0.9117 accuracy but zero recall, illustrating why we prioritize positive-class F1 instead of accuracy alone.

A 1,000-replicate multinomial bootstrap of holdout confusion categories (seed 7) gives a percentile F1 interval of [0.4348, 0.4549]. This interval is conditional on the fitted model and fixed split; it does not account for training, model selection or reuse of the holdout.

4 Reproducibility and limitations

After freezing the configuration, tree count and cutoff, we retrain all trees and bin edges from all labeled rows. This final model produces the supplied 109,379 test predictions, of which 14,410 are positive. The table evaluates the smaller training-partition models, not this final full-data model. No hidden-test score or AIcrowd submission is claimed.

Training and inference import only NumPy and standard Python modules. The saved model is human-readable JSON containing parameters, cuts, base logit and trees. `run.py` checks input and model hashes and feature order, then recreates the CSV without retraining. The package records data hashes, split IDs, tuning/holdout probabilities, model settings and verification results. The runtime is Python 3.12 with NumPy 2.3.5.

Unit tests cover analytic leaf values, missing observations, training-only cuts and exact save/load predictions. Independent checks compare the split calculation against direct enumeration on 300 random cases and exercise an end-to-end synthetic training/inference round trip. All six required functions in `implementations.py` use the specified signatures, one-dimensional weights, batch-size-one SGD and unpenalized returned losses. MSE includes the factor 1/2, and least squares uses normal equations without `numpy.linalg.lstsq`.

Survey categories remain numeric, and survey identifiers other than the explicit CSV ID are retained. Empty fields and NaNs are treated as missing, but questionnaire-specific nonresponse codes are not fully decoded. We do not use survey weights or geographic/temporal validation. The target is reported disease history; these results do not validate future clinical risk prediction. Quantile bins and shallow trees also limit expressiveness. A small F1 difference between candidates should not be interpreted as robust superiority from one split.

5 Course requirements and provenance

This package satisfies the stated library restriction and includes reproducible inference and the six required algorithms. The foundation module incorporates a teammate's implementations, with formatting and a singular-matrix fallback; `base_models.py` provides shared training and prediction entry points. The original teammate file is preserved. Formal delivery still requires team names, a public GitHub commit URL and online submission. Integration, boosting code and writing used AI assistance; the team should verify any additional course policy governing that assistance.

[1] EPFL, Project 1 materials and supplied Fall 2026 description.

[2] Introduction to Boosted Trees: mathematical objective and split derivation; no implementation copied.

[3] AIcrowd challenge. Sources accessed 28 October 2026.